

Multi-Agent AI Systems: A Complete Guide

Understanding LangGraph, RAG, MCP, and Claude Sonnet

Chapter 1: Introduction to Multi-Agent AI

Multi-agent AI systems represent a paradigm shift in how we build intelligent applications. Instead of relying on a single monolithic model to handle all tasks, multi-agent systems decompose complex problems into specialized sub-tasks, each handled by a dedicated agent. This approach improves modularity, scalability, and the overall quality of responses.

A multi-agent system typically consists of a router or orchestrator that receives user input and directs it to the most appropriate agent. Each agent has a specific skill set — some agents excel at retrieving information from documents (RAG agents), while others are better at executing tools like calculators or web APIs (tool agents).

The key benefits of multi-agent architectures include separation of concerns, parallel execution of tasks, specialization of agents, fault isolation, and easier debugging. Each agent can be developed, tested, and improved independently without affecting others.

Chapter 2: LangGraph — The Orchestration Framework

LangGraph is a library built on top of LangChain that enables the creation of stateful, multi-agent workflows using a graph-based approach. In LangGraph, the application is modeled as a directed graph where nodes represent processing steps and edges define the flow of data.

The core concepts in LangGraph are: State (a shared TypedDict that flows through the entire graph), Nodes (Python functions that receive state and return updated state), Edges (connections between nodes, either fixed or conditional), and the Compiled Graph (the runnable workflow).

Conditional edges are particularly powerful in LangGraph. They allow the graph to make routing decisions at runtime based on the current state. For example, a router node can examine the user query, determine whether it requires document retrieval or tool use, and route accordingly. LangGraph maintains full state persistence across nodes, meaning every agent in the pipeline has access to the complete history of what happened before it.

Chapter 3: RAG — Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a technique that enhances LLM responses by grounding them in external knowledge. Instead of relying solely on the model's training data, RAG retrieves relevant documents at query time and includes them in the prompt as context.

The RAG pipeline consists of two main phases. The Indexing Phase involves loading documents from a source such as PDF or web, splitting them into smaller chunks, generating vector embeddings for each chunk using a sentence transformer model, and storing those vectors in a vector database like ChromaDB. This phase runs once and the results are persisted to disk.

The Retrieval Phase runs at query time. The user's question is embedded using the same model, and a similarity search is performed against the vector database to find the most semantically relevant chunks. These chunks are then injected into the LLM prompt as context, enabling the model to produce accurate, grounded answers with citations.

ChromaDB is an open-source vector database designed specifically for AI applications. It supports cosine similarity search, metadata filtering, and persistent storage. The all-MiniLM-L6-v2 model from Sentence Transformers produces 384-dimensional embeddings and strikes an excellent balance between speed and retrieval quality.

Chapter 4: MCP — Model Context Protocol

The Model Context Protocol (MCP) is an open standard developed by Anthropic that defines how language models communicate with external tools and data sources. MCP provides a standardized interface so that any LLM can discover and invoke any MCP-compatible tool without custom integration code for each pair.

An MCP server exposes tools by declaring their name, description, and input schema in a standardized JSON format. The LLM client such as Claude reads these declarations and can decide to invoke any tool by generating a `tool_use` message. The MCP server receives the invocation, executes the tool, and returns a `tool_result`.

MCP supports multiple transport mechanisms including stdio (standard input/output for subprocess communication) and HTTP with Server-Sent Events for networked services. The stdio transport is ideal for local single-user applications, while HTTP transport enables multi-client distributed architectures.

Chapter 5: Claude Sonnet — The Language Model

Claude Sonnet is Anthropic's mid-tier model in the Claude family, optimized for an excellent balance between intelligence and speed. It supports a 200,000-token context window, making it ideal for long document analysis. Claude Sonnet natively supports tool use — the model can reason about which tool to call and format the invocation correctly.

Anthropic's tool use API follows a two-turn pattern. In the first turn, you send the user message along with tool definitions. Claude responds with a `tool_use` block specifying which tool to call and with what arguments. In the second turn, you send the `tool_result` back, and Claude generates the final natural-language response incorporating the result.

Claude's safety architecture ensures that it refuses harmful requests, maintains honesty about uncertainty, and avoids hallucination where possible. This makes it particularly well-suited for RAG applications where accuracy and citation are critical.

Chapter 6: Sentence Transformers and Vector Embeddings

Sentence Transformers are neural network models that convert text into dense numerical vectors (embeddings) that capture semantic meaning. Two texts with similar meaning will have vectors that are geometrically close in the embedding space, regardless of the specific words used. This property enables semantic search.

The all-MiniLM-L6-v2 model produces 384-dimensional embeddings and is one of the most popular choices for RAG applications due to its small size of 80 MB, fast inference on CPU, and strong performance on semantic similarity benchmarks. It is fully open-source and requires no API key or internet connection after the initial download.

Vector similarity is typically measured using cosine similarity, which computes the angle between two vectors. A score of 1.0 means identical direction (very similar texts), while 0.0 means perpendicular (unrelated texts). ChromaDB uses this metric to rank retrieved chunks from most to least relevant.

Chapter 7: Technology Stack Summary

Component	Technology	Purpose
LLM	Claude Sonnet 4-6	Text generation and reasoning
Orchestration	LangGraph 0.2.x	Agent routing and workflow
RAG Framework	LangChain 0.2.x	Document loading and chaining
Vector DB	ChromaDB 0.5.x	Embedding storage and retrieval
Embeddings	Sentence Transformers	Text-to-vector conversion
Tool Protocol	MCP 1.x	Standardized tool interface
UI	Streamlit 1.38	Web-based chat interface